

P ,NP Problems

Module-7

COMPUTATIONAL COMPLEXITY CLASS

- Problem are classified into two types based on the hardness
- **Analytical hard problems**
 - Do not have a proper mathematical formulation
 - Need enormous amount of information
 - Incomplete specifications,, partial problem statements etc.
- **Computational hard problems**
 - Intractable problems that cannot be solved using available algorithms that use a available resources.
 - Are categorized into two groups called **complexity classes**.
- **Complexity classes** - set of problem s that have similar runtime or rates growth
- Classified as **Class P** and **Class NP**

P, NP Problems

Definition of P class Problem:

- The set of decision-based problems come into the division of P Problems who can be solved or produced an output within polynomial time. P problems being easy to solve.
- Class P are considered easy problems, can be computed efficiently.
- E.g. searching, sorting and selection problems.

Definition of NP class Problem: -

- The set of all decision-based problems came into the division of NP Problems who can't be solved or produced an output within polynomial time but verified in the **polynomial time**.
- NP class contains P class as a subset. NP problems being hard to solve.
- Problems are intractable and do not have any efficient solution.

Definition of Polynomial time:

- If we produce an output according to the given input within a specific amount of time such as within a minute, hours. This is known as Polynomial time.

Definition of Non-Polynomial time:

If we produce an output according to the given input but there are no time constraints is known as Non-Polynomial time. But yes output will produce but time is not fixed yet.

Example for NP Problem

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Types of NP

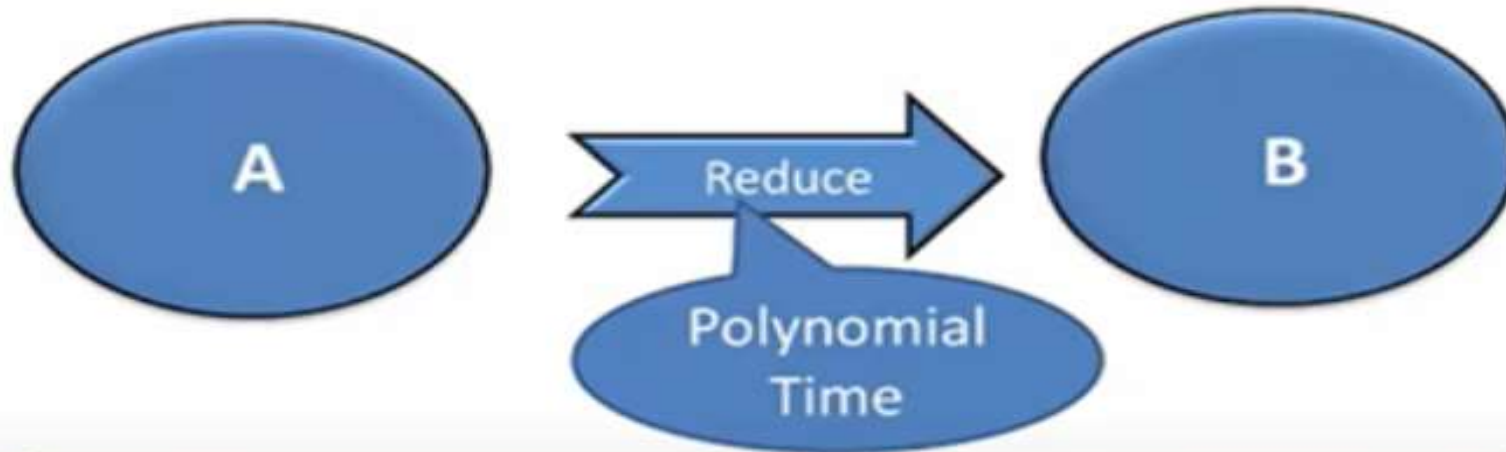
- NP hard problems
- NP Complete problems

NP-hard

- **Definition of NP-hard class:** - Here you to satisfy the following points to come into the division of NP-hard
 1. If we can solve this problem in polynomial time, then we can solve all NP problems in polynomial time
 2. If you convert the issue into one form to another form within the polynomial time

Reduction

Reduction:



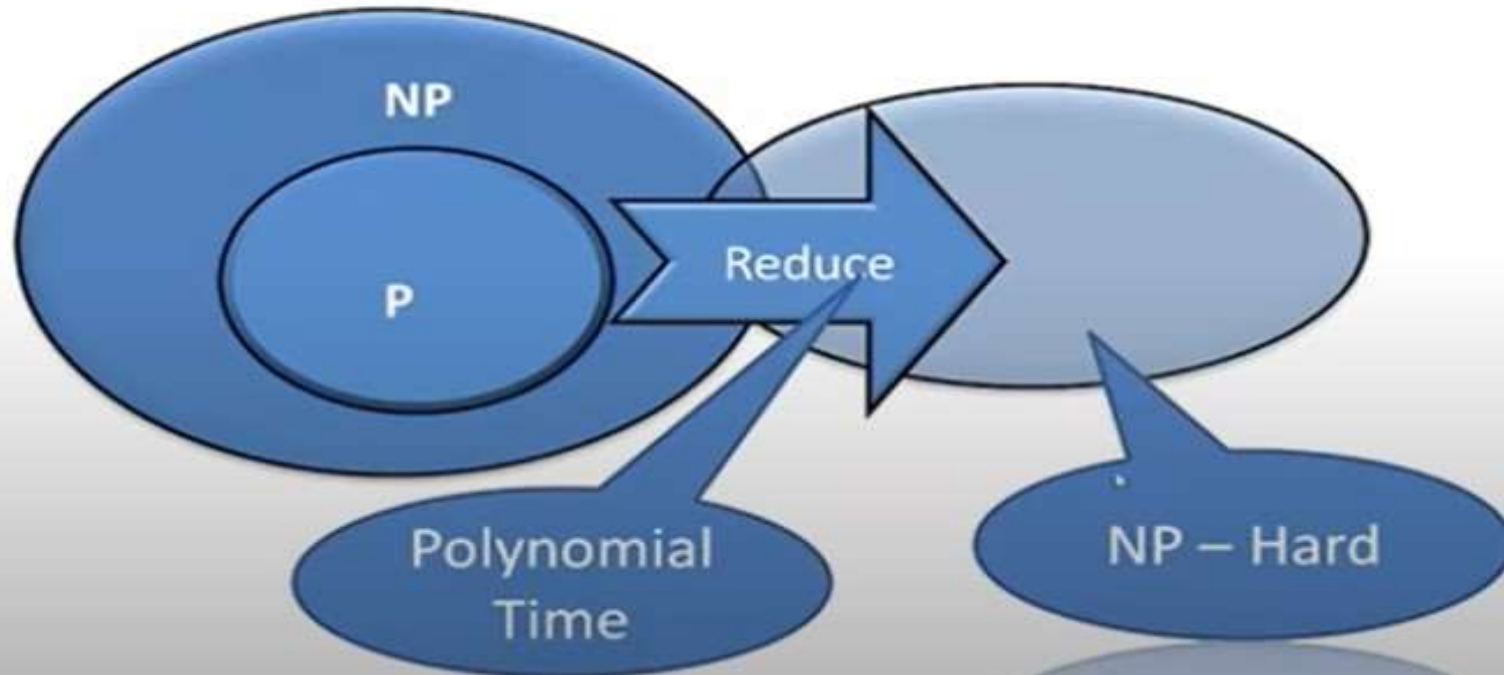
Let A and B are two problems then problem A reduces to problem B iff there is a way to solve A by deterministic algorithm that solve B in polynomial time.

If A is reducible to B we denote it by $A \leq B$

NP Hard

NP Hard Problem:

A Problem is NP-Hard if every problem in NP can be polynomial reduced to it.



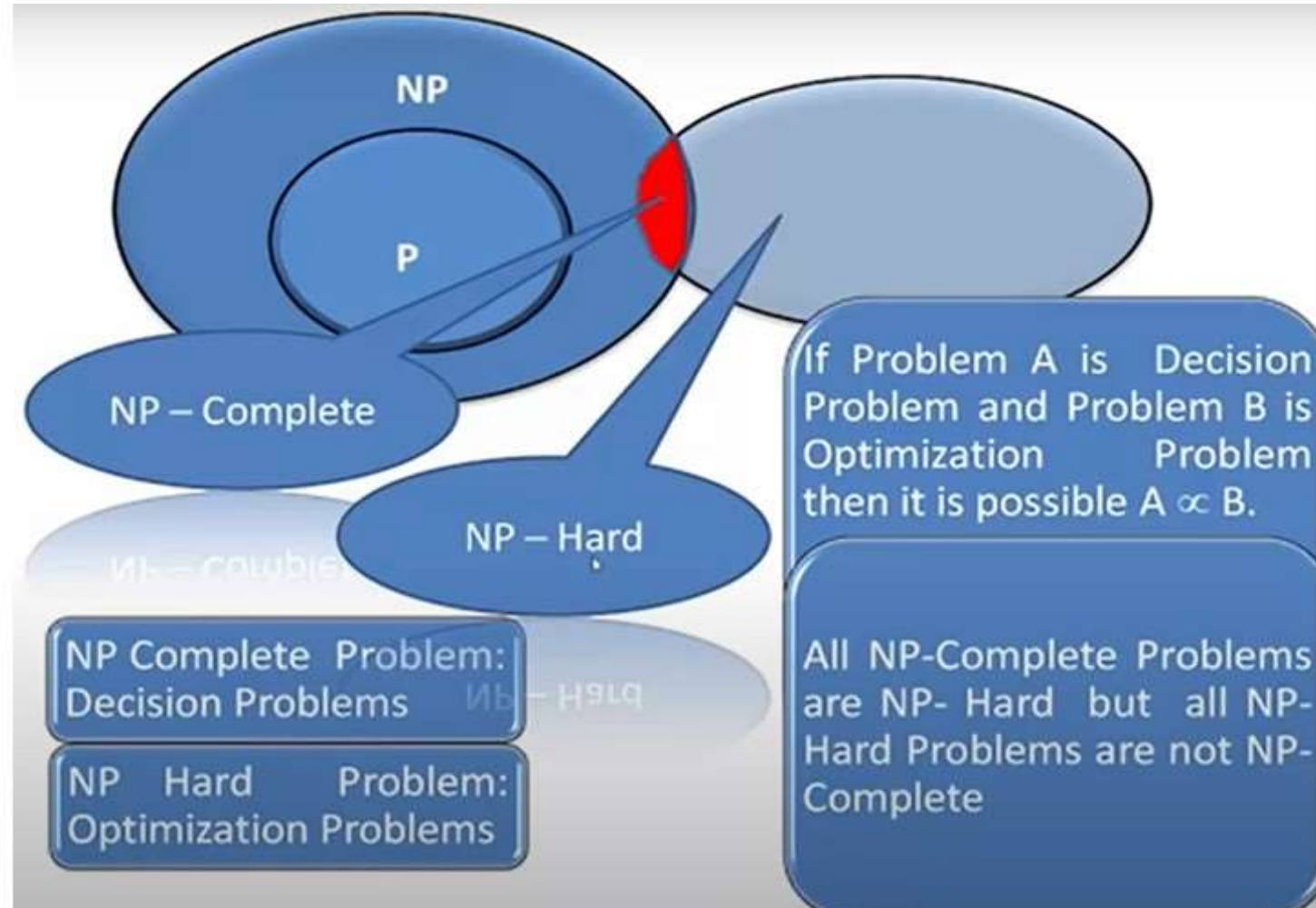
NP-complete

Definition of NP-complete class: -

A problem is in NP-complete, if

1. It is in NP
2. It is NP-hard

NP COMPLETE



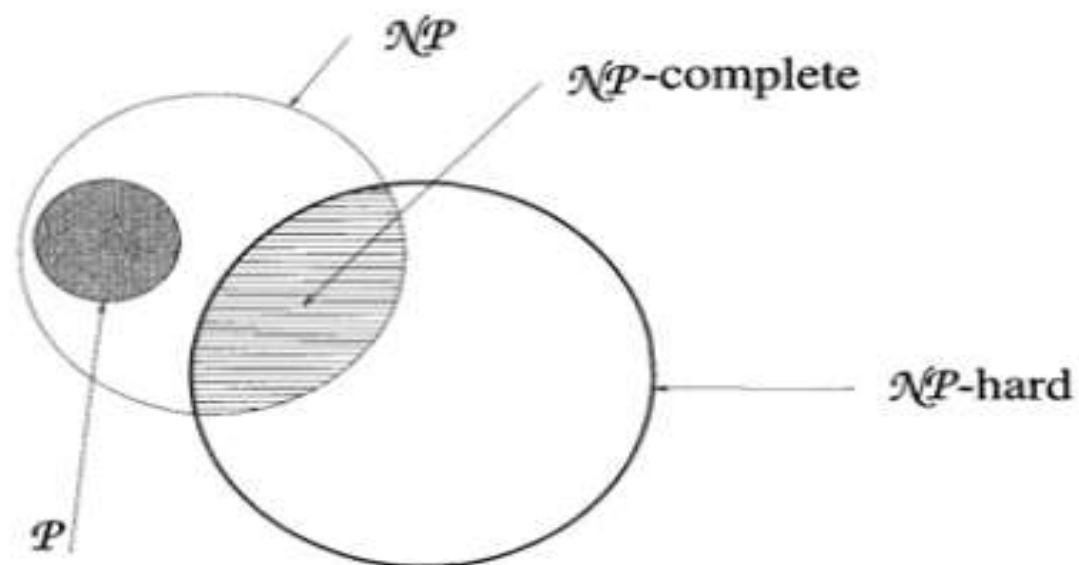
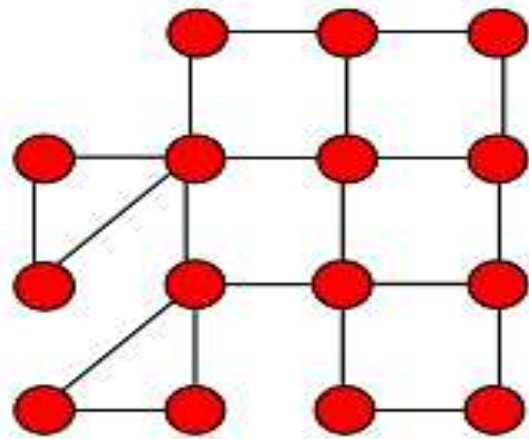


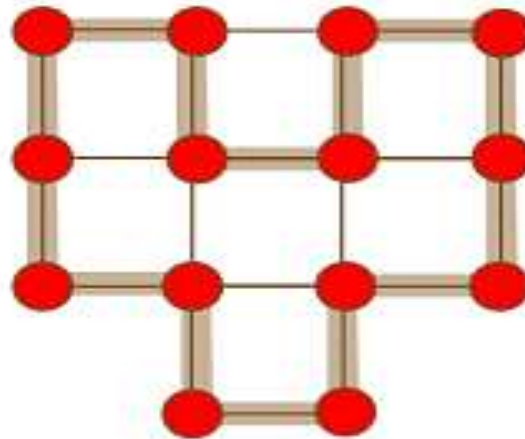
Figure 11.2 Commonly believed relationship among $\mathcal{P}$, $\mathcal{NP}$, $\mathcal{NP}$ -complete, and $\mathcal{NP}$ -hard problems

Polynomial Time Verification

- Many problems are hard to solve, but they have the property that it is easy to authenticate the solution if one is provided.
- **Hamiltonian cycle problem:-**
- Consider the Hamiltonian cycle problem. Given an undirected graph G , does G have a cycle that visits each vertex exactly once? There is no known polynomial time algorithm for this dispute.



Non-Hamiltonian



Hamiltonian

- We could then inspect the graph and check that this is indeed a legal cycle and that it visits all of the vertices of the graph exactly once.
- Thus, even though we know of no efficient way to solve the Hamiltonian cycle problem, there is a beneficial way to verify that a given cycle is indeed a Hamiltonian cycle.

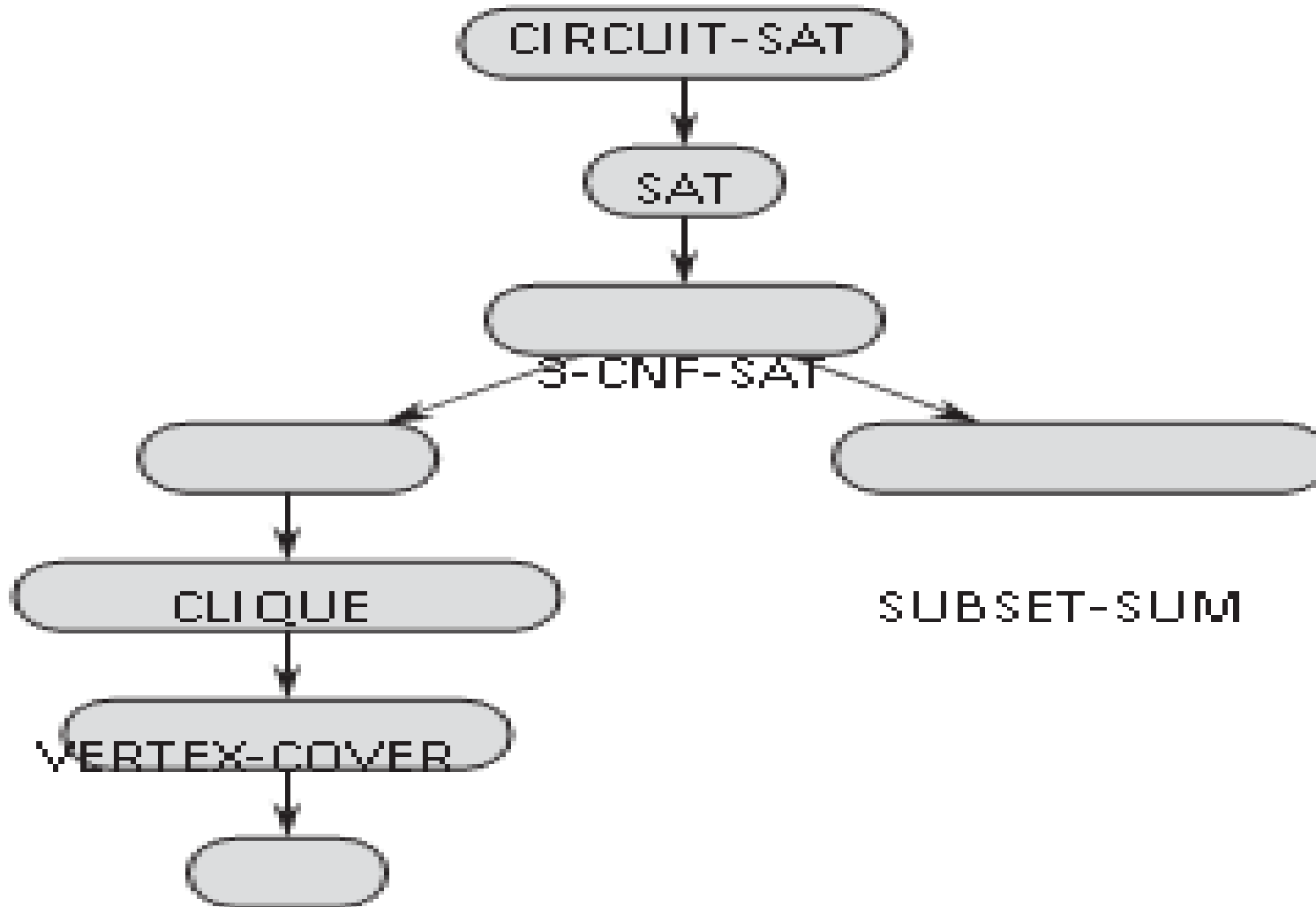
Relation of P and NP classes

1. P contains in NP

2. $P=NP$

- Observe that P contains in NP. In other words, if we can solve a problem in polynomial time, we can indeed verify the solution in polynomial time.
- More formally, we do not need to see a certificate (there is no need to specify the vertex/intermediate of the specific path) to solve the problem; we can explain it in polynomial time anyway.
- However, it is not known whether $P = NP$. It seems you can verify and produce an output of the set of decision-based problems in NP classes in a polynomial time which is impossible because according to the definition of NP classes you can verify the solution within the polynomial time.
- So this relation can never be held.

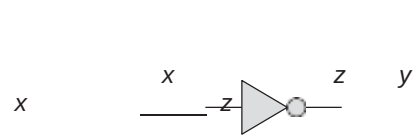
SATISFIABILITY PROBLEM (SAT)



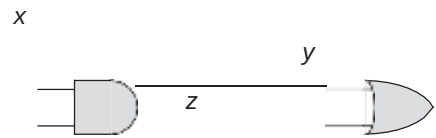
Circuit satisfiability

- Boolean combinational circuits are built from boolean combinational elements that are interconnected by wires.
- A ***boolean combinational element*** is any circuit element that has a constant number of boolean inputs and outputs and that performs a well-defined function
- Boolean values are drawn from the set $\{0,1\}$, where 0 represents FALSE and 1 represents TRUE.
- The boolean combinational elements that we use in the circuit-satisfiability problem compute simple boolean functions, and they are known as ***logic gates***.

- The three basic logic gates that we use in the circuit-satisfiability problem: the NOT gate (or inverter), the AND gate, and the OR gate.
- The NOT gate takes a single binary input x , whose value is either 0 or 1, and produces a binary output x' whose value is opposite that of the input value.
- Each of the other two gates takes two binary inputs x and y and produces a single binary output z .
- the operation of each gate, and of any boolean combinational element, is described by a **truth table**.
- A truth table gives the outputs of the combinational element for each possible setting of the inputs.



x	x
0	1
1	0

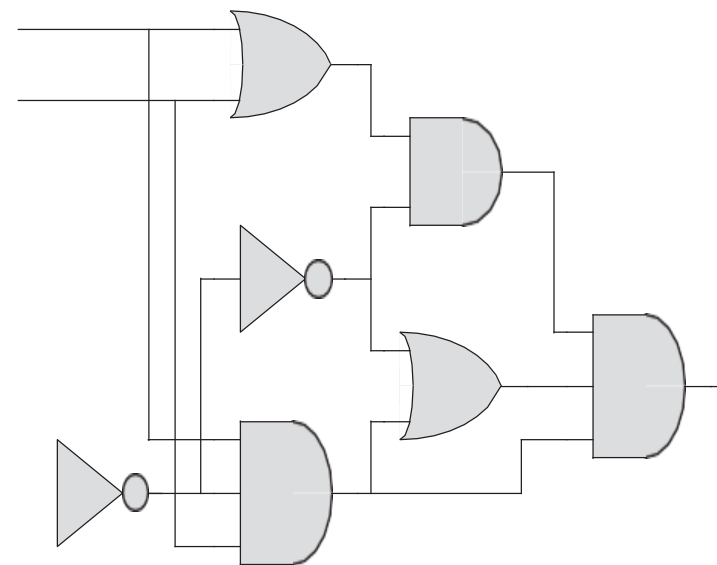
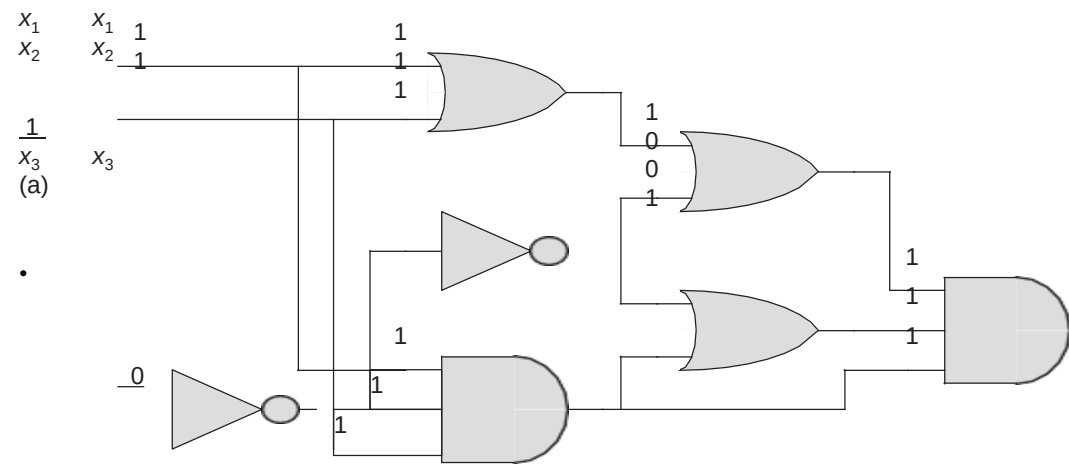


x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

x	y	$x \vee y$
0	0	0
0	1	1
1	0	1
1	1	1

- We use the symbols $\neg$ to denote the NOT function, $\wedge$ to denote the AND function, and $\vee$ to denote the OR function. Thus, for example, $0 \vee 1 = 1$
- An AND gate's output is 1 if all of its inputs are 1, and its output is 0 otherwise. An OR gate's output is 1 if any of its inputs are 1, and its output is 0 otherwise
- A boolean combinational circuit consists of one or more boolean combinational elements interconnected by wires.
- A wire can connect the output of one element to the input of another, thereby providing the output value of the first element as an input value of the second.

- The number of element inputs fed by a wire is called the ***fan-out*** of the wire.
- If no element output is connected to a wire, the wire is a ***circuit input***, accepting input values from an external source.
- If no element input is connected to a wire, the wire is a ***circuit output***, providing the results of the circuit's computation to the outside world.
- A ***truth assignment*** for a boolean combinational circuit is a set of boolean input values.
- one-output boolean combinational circuit is ***satisfiable*** if it has a ***satisfying assignment***: a truth assignment that causes the output of the circuit to be 1.



SATISFIABILITY PROBLEM (SAT)

- Stephen Arthur Cook and L.A. Levin in 1973 proved that the **satisfiability problem (SAT)** is NP-complete.
- **SAT is a significant problem and can be stated as follows**
 - Given Boolean expression F having n variables $x_{11}, x_{22}, \dots, x_n$ and Boolean operators, is it possible to have assignment for variables true or false such that the binary expression F is true?
 - This problem is known as **formula-SAT**.
 - A SAT takes a Boolean expression F and checks whether the given expression is satisfiable.
 - A Boolean expression is said to be satisfiable for some valid variables.

SATISFIABILITY PROBLEM

➤ Terminologies

- **Boolean Variable** – say x , that can have **only two values** (true or false)
- **Literal** – logical variable x is called positive and **negation of x** is called negative.
- **Clause** – Sequence of variable $x_{11}, x_{22}, \dots, x_n$ can be **separated by logical OR operator**.

SATISFIABILITY PROBLEM

➤ Terminologies

- **Expression** – combine all the preceding clauses using a Boolean operator to form an expression.
- **CNF form** – An expression is in Conjunctive Normal Form if set of clause are separated by AND operator, while literal are connected by OR operator. $f = (x_1 \vee \overline{x_2} \vee x_3) \wedge (x_1 \vee \overline{x_3} \vee x_2)$
- **3-CNF** – an expression is said to be in 3-CNF if it is in the conjunctive normal form, and every clause has exact three literals.

SATISFIABILITY PROBLEM

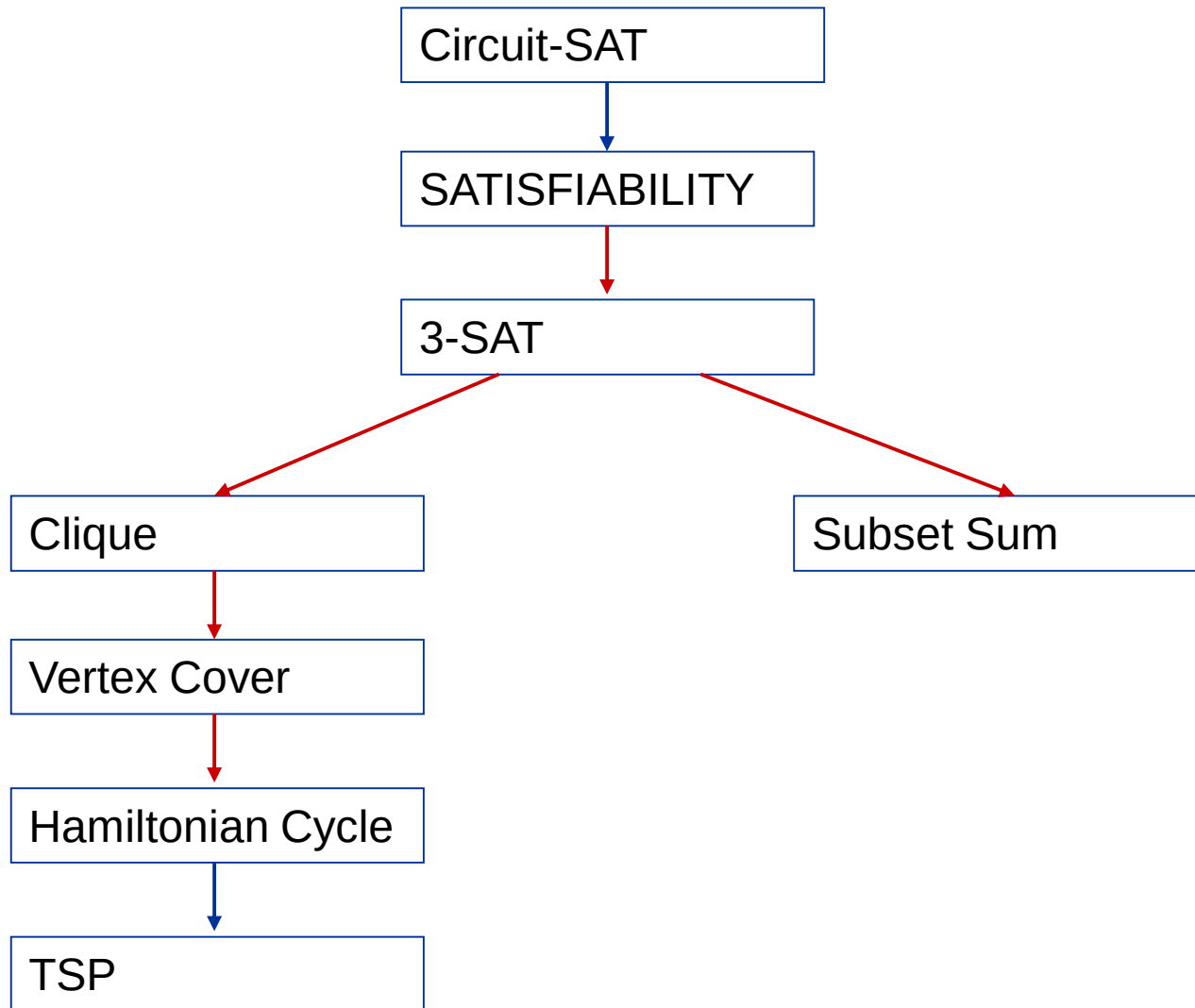
- A general definition of the SAT problem.
- A given Boolean expression, determining **if there exists a truth assignment to its variables for which the expression is true is defined as the satisfiability problem.**
- Let's take an example to discuss further.

a	b	c	$(a \wedge b) \vee c$	$(a \wedge \neg a) \vee (c \wedge \neg c)$
1	1	1	1	0
0	1	1	1	0
0	0	1	1	0
0	1	0	0	0
1	0	1	1	0
1	0	0	0	0
1	0	1	1	0
0	0	0	0	0

SATISFIABILITY PROBLEM

- To test whether there exists a truth assignment to variables a, b, c , we've tested all possible assignments.
- There are 5 satisfying truth assignments for the expression $(a \wedge b) \vee c$ and hence it is satisfiable.
- But the expression $(a \wedge \neg a) \vee (c \wedge \neg c)$ is false for all possible truth assignments and hence it is unsatisfiable.
- This is called the **truth table method**

a	b	c	$(a \wedge b) \vee c$	$(a \wedge \neg a) \vee (c \wedge \neg c)$
1	1	1	1	0
0	1	1	1	0
0	0	1	1	0
0	1	0	0	0
1	0	1	1	0
1	0	0	0	0
1	0	1	1	0
0	0	0	0	0



3-CNF satisfiability or 3-CNF-SAT

Define 3-CNF satisfiability using the following terms.

- A literal in a boolean formula is an occurrence of a variable or its negation.
- A boolean formula is in **conjunctive normal form**, or CNF, if it is expressed as an AND of clauses, each of which is the OR of one or more literals.
- A boolean formula is in **3-conjunctive normal form**, or 3-CNF, if each clause has exactly three distinct literals.

For example, the boolean formula

$$(x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_3 \vee x_2 \vee x_4) \wedge (\neg x_1 \vee \neg x_3 \vee \neg x_4)$$

is in 3-CNF. The first of its three clauses is $(x_1 \vee \neg x_1 \vee \neg x_2)$, which contains the three literals x_1 , $\neg x_1$, and $\neg x_2$.

Satisfiability of boolean formulas in 3-conjunctive normal form is NP-complete.

Proof:

- To prove , we need only show that $\text{SAT} \leq_p \text{3-CNF-SAT}$.
- We break the reduction algorithm into three basic steps.
- Each step progressively transforms the input formula closer to the desired 3-conjunctive normal form.

- First, we construct a binary “parse” tree for the input formula , with literals as leaves and connectives as internal nodes
- For example,
- Consider the formula

$$\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2 .$$

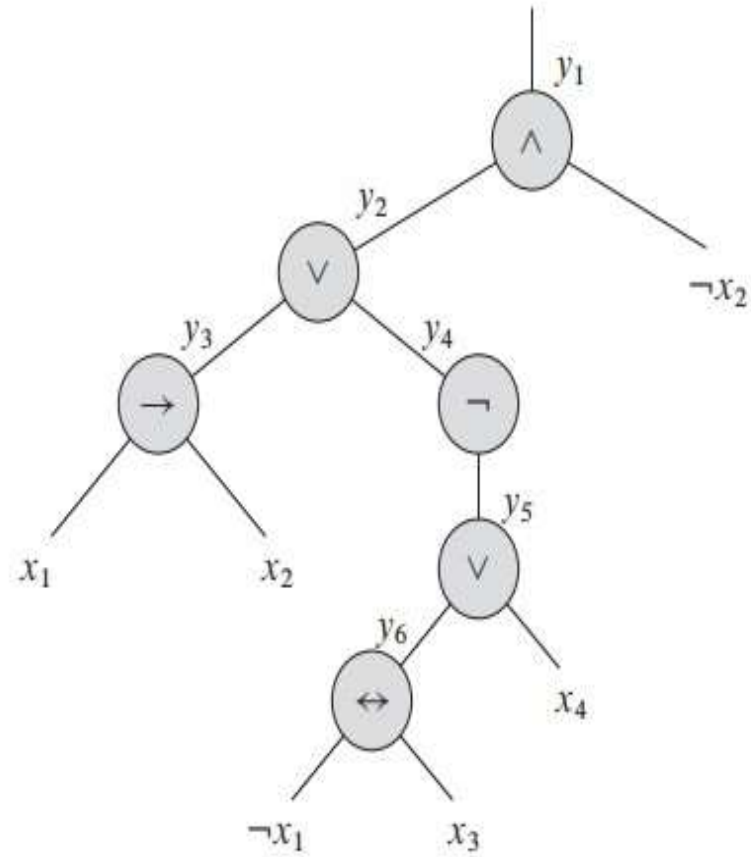


Figure 34.11 The tree corresponding to the formula $\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$.

- we introduce a variable y_i for the output of each internal node. Then, we rewrite the original formula as the AND of the root variable and a conjunction of clauses describing the operation of each node

$$\begin{aligned}\phi' = & y_1 \wedge (y_1 \leftrightarrow (y_2 \wedge \neg x_2)) \\ & \wedge (y_2 \leftrightarrow (y_3 \vee y_4)) \\ & \wedge (y_3 \leftrightarrow (x_1 \rightarrow x_2)) \\ & \wedge (y_4 \leftrightarrow \neg y_5) \\ & \wedge (y_5 \leftrightarrow (y_6 \vee x_4)) \\ & \wedge (y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3)) .\end{aligned}$$

Observe that the formula ϕ' thus obtained is a conjunction of clauses ϕ'_i , each of which has at most 3 literals. The only requirement that we might fail to meet is that each clause has to be an OR of 3 literals.

Step:2

The second step of the reduction converts each clause ϕ'_i into conjunctive normal form. We construct a truth table for ϕ'_i by evaluating all possible assignments to its variables. Each row of the truth table consists of a possible assignment of the variables of the clause, together with the value of the clause under that assignment. Using the truth-table entries that evaluate to 0, we build a formula in *disjunctive normal form* (or *DNF*)—an OR of ANDs—that is equivalent to $\neg\phi'_i$. We then negate this formula and convert it into a CNF formula ϕ''_i by using *DeMorgan's*

laws for propositional logic,

$$\neg(a \wedge b) = \neg a \vee \neg b ,$$

$$\neg(a \vee b) = \neg a \wedge \neg b ,$$

- TRUTH TABLE

y_1	y_2	x_2	$(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	1

Figure 34.12 The truth table for the clause $(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$.

Conditional:		
p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Biconditional:		
p	q	$p \leftrightarrow q$
T	T	T
T	F	F
F	T	F
F	F	T

In our example, we convert the clause $\phi'_1 = (y_1 \leftrightarrow (y_2 \wedge \neg x_2))$ into CNF as follows. The truth table for ϕ'_1 appears in Figure 34.12. The DNF formula equivalent to $\neg\phi'_1$ is

$$(y_1 \wedge y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge \neg x_2) \vee (\neg y_1 \wedge y_2 \wedge \neg x_2) .$$

Negating and applying DeMorgan's laws, we get the CNF formula

$$\begin{aligned} \phi''_1 = & (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2) \\ & \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2) , \end{aligned}$$

which is equivalent to the original clause ϕ'_1 .

At this point, we have converted each clause ϕ'_i of the formula ϕ' into a CNF formula ϕ''_i , and thus ϕ' is equivalent to the CNF formula ϕ'' consisting of the conjunction of the ϕ''_i . Moreover, each clause of ϕ'' has at most 3 literals.

The third and final step of the reduction further transforms the formula so that each clause has *exactly* 3 distinct literals. We construct the final 3-CNF formula ϕ''' from the clauses of the CNF formula ϕ'' . The formula ϕ''' also uses two auxiliary variables that we shall call p and q . For each clause C_i of ϕ'' , we include the following clauses in ϕ''' :

- If C_i has 3 distinct literals, then simply include C_i as a clause of ϕ''' .
- If C_i has 2 distinct literals, that is, if $C_i = (l_1 \vee l_2)$, where l_1 and l_2 are literals, then include $(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \neg p)$ as clauses of ϕ''' . The literals p and $\neg p$ merely fulfill the syntactic requirement that each clause of ϕ''' has exactly 3 distinct literals. Whether $p = 0$ or $p = 1$, one of the clauses is equivalent to $l_1 \vee l_2$, and the other evaluates to 1, which is the identity for AND.

If C_i has just 1 distinct literal l , then include $(l \vee p \vee q) \wedge (l \vee p \vee \neg q) \wedge (l \vee \neg p \vee q) \wedge (l \vee \neg p \vee \neg q)$ as clauses of ϕ''' . Regardless of the values of p and q , one of the four clauses is equivalent to l , and the other 3 evaluate to 1.

Theorem: 3-SAT is NP-complete

Proof.

Step 1: Select suitable NP-hard problem: Circuit-Sat or SATISFIABILITY ?

SATISFIABILITY

Step 2: Give polynomial-time reduction from SATISFIABILITY to 3-SAT ✓

Convert arbitrary Boolean formula F into formula F^* in 3-CNF form

Step 3. Prove 3-SAT is in NP by giving polynomial-time verification algorithm. ■

certificate = satisfying assignment